FIME / UANL

REPORTE TÉCNICO DE LABORATORIO

UNIDAD 5 · Integración, Pruebas y Calidad

Componentes de Lógica de Negocios · Proyecto Python de pedidos

INDICADOR	RESULTADO
Pruebas automatizadas	37 de 37 aprobadas
Cobertura sobre src/	100 % de líneas medidas
Evidencia del ciclo TDD	RED reproducida / GREEN aprobada
Pipeline en GitHub	Preparado; ejecución remota pendiente
Adaptación al PIA	Requiere reglas reales de la empresa

Alumno(a): _____

Matrícula: _____ Grupo: _____

Docente: _____

Fecha: 4 de noviembre de 2026

Documento editable con evidencias verificables y espacios identificados para los componentes que dependen del alumno.

1. Resumen del trabajo y alcance

El laboratorio verifica las reglas de negocio de un sistema didáctico de pedidos. Las pruebas cubren confirmación, descuento VIP del 15 %, rechazo de importes no positivos o clientes con deuda, autorización de cancelación, persistencia en SQLAlchemy y simulación de cobros mediante mocks. Se usa SQLite en memoria para no modificar datos de producción.

La versión evaluada corresponde al proyecto CLN_U5_Laboratorio_Resuelto, ampliado con cuatro pruebas expresamente solicitadas por la guía. En un entorno Linux con Python 3.13.5 y pytest 9.0.2 se ejecutaron 37 pruebas: todas aprobaron, con 100 % de cobertura de líneas de src/. Se generó el reporte navegable htmlcov/index.html.

1.1 Estado de las evidencias

CLAVE	REQUISITO DE LA GUÍA	ESTADO VERIFICABLE
E1	Preparación y salida de pytest	Registro local disponible; captura de PC opcional
E2	Pruebas 200 VIP y regular	Implementadas y aprobadas
E3	Ciclo RED → GREEN → REFACTOR	RED reproducida desde base; GREEN verificada
E4	Commit, rollback y pedido id=2	Pruebas aprobadas
E5	Mocks; rechazo VIP 200 / cargo 170	Prueba aprobada y riesgo descrito
E6	Política de cancelación y límite 24 h	Pruebas aprobadas y matriz incluida
E7	Cobertura HTML ≥80 %; Ruff	100 % generado; Ruff pendiente de validar
E8	CI verde → roja → verde	Workflow creado; faltan ejecuciones en GitHub
E9	Tres reglas del PIA / mínimo 9 casos	Plantilla incluida; reglas reales no facilitadas

1.2 Qué se entrega

Este reporte, el proyecto completo, las pruebas originales y complementarias, MATRIZ_RESULTADOS.csv, las salidas reales de pytest en evidencias/E1-E7/, el HTML de cobertura y la configuración .github/workflows/ci.yml. Los datos CSV y el archivo SQLite adjunto son ficticios y el SQLite didáctico no se importa automáticamente al repositorio de pruebas.

LÍMITE DE VERIFICACIÓN No fue posible ejecutar Ruff dentro de este entorno aislado por no estar instalado. Tampoco se dispone de un repositorio GitHub del alumno ni de las reglas aprobadas de su PIA; estos puntos permanecen pendientes y no se presentan como concluidos.

2. Entorno de trabajo y método

COMPONENTE	DESCRIPCIÓN
Lenguaje	Python 3.13.5 en validación; la guía pide ≥3.11
Pruebas	pytest 9.0.2 y pytest-cov
Persistencia	SQLAlchemy 2.x con SQLite en memoria
Simulación externa	unittest.mock.Mock; no se realizaron pagos reales
Calidad / integración	Ruff (pendiente de comprobación) y GitHub Actions configurado
Alcance E2E	Escenario propuesto; no hay API ni interfaz gráfica en el proyecto base

```
python -m pip install -r requirements.txt
python -m pytest -v
python -m ruff check src tests cargar_datos_demo.py
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

pytest descubre y ejecuta funciones de prueba que comienzan por test_; el entorno virtual (venv) aísla dependencias de este laboratorio respecto a otros proyectos. El patrón Arrange–Act–Assert organiza cada verificación en preparación, acción y comprobación.

3. Evidencias E1 y E2 · Ejecución y pruebas unitarias

E1. Instalación y primera ejecución

RESULTADO Suite ampliada: 37 aprobadas en 0.40 s; ejecución local en Linux. El registro completo puede leerse en evidencias/E1-E7/E1_E7_suite_cobertura.log.



Visualización del registro de consola generado con salida real; no es captura de la PC del alumno.

E2. Reglas de total y descuento

ENTRADA	REGLA	ESPERADO	OBSERVADO
Total=200; VIP=True	Descuento de 15 %	Cobro 170; CONFIRMED	170; CONFIRMED
Total=200; VIP=False	Sin descuento	Cobro 200; CONFIRMED	200; CONFIRMED
Total=100; deuda=50	Deuda bloquea confirmación	CustomerHasDebt	CustomerHasDebt
Total=0 o -1	Importe positivo obligatorio	ValueError	ValueError

Las dos pruebas explícitas de total 200 están en tests/test_evidencias_guia.py. No requieren base de datos porque Order.confirm() aplica reglas de dominio en memoria. Tras las comprobaciones se conserva el estado CONFIRMED para los casos válidos.

Ejemplo Arrange–Act–Assert

```
order = Order(200, vip=True)   # Arrange
amount = order.confirm()     # Act
assert amount == 170         # Assert
assert order.status == "CONFIRMED"
```

4. Evidencia E3 · TDD Red–Green–Refactor

Se ejecutó la prueba de doble confirmación contra una copia intacta del ZIP base: falló porque la implementación inicial permitía volver a confirmar el mismo pedido. Esa falla constituye la fase RED; no se obtuvo falseando una aserción de la versión final.

E3 / RED — reproducción sobre código base

```
$ python -m pytest tests/test_tdd_red.py -v
tests/test_tdd_red.py::test_cannot_confirm_twice FAILED [100%]
> with pytest.raises(ValueError, match="already confirmed"):
E Failed: DID NOT RAISE <class 'ValueError'>
==================== 1 failed in 0.05s ====================
```

RED: ejecución real sobre el código base original; salida completa en E3_red_codigo_base.log.

En la versión corregida, Order.confirm() verifica primero el estado del pedido. Si ya es CONFIRMED, lanza OrderAlreadyConfirmed("order already confirmed"); la prueba actualizada espera la excepción específica y verifica que el estado y el descuento no cambian.

```
if self.status == "CONFIRMED":
    raise OrderAlreadyConfirmed("order already confirmed")
```

E3 / GREEN — versión corregida

```
$ python -m pytest tests/test_csv_cases.py::test_second_confirmation_rejected_after_first_confi…
tests/test_csv_cases.py::test_second_confirmation_rejected_after_first_confirmation PASSED
==================== 1 passed in 0.03s ====================
```

GREEN: la prueba de segunda confirmación se aprueba en la versión corregida.

REFACTOR: se conserva una excepción de dominio específica en vez del ValueError ilustrativo de la guía, sin cambiar la regla ni el mensaje. TDD consiste en definir primero la expectativa, observar RED, alcanzar GREEN y mejorar el diseño sin modificar la conducta comprobada.

5. Evidencias E4 y E5 · Persistencia y servicios externos

E4. SQLAlchemy: commit y rollback

PRUEBA	OPERACIÓN	RESULTADO REAL
CP14	Guardar registro y ejecutar commit	Visible en una nueva sesión
CP15	Guardar, flush y ejecutar rollback	No visible tras reabrir sesión
Caso guía extra	Guardar id=2, total=250 y ejecutar commit	id=2, total=250, status=NEW

commit finaliza la transacción guardando los cambios en la base de pruebas; rollback deshace las modificaciones pendientes de la transacción. Los casos usan SQLite en memoria y sesiones aisladas para no tocar información real. El registro de ejecución se incluye en E4_persistencia.log.

E5. Pasarela de pago simulada con Mock

ESCENARIO	COMPORTAMIENTO ESPERADO	RESULTADO
Aprobado; total=100	charge(100)=True; devuelve 100	PASSED
Rechazado; VIP=200	charge(170)=False; RuntimeError	PASSED
Cargo real	Ninguno: el gateway se simula	No se realizan pagos

```
gateway = Mock()
gateway.charge.return_value = False
order = Order(200, vip=True)
with pytest.raises(RuntimeError, match="payment rejected"):
    OrderService(gateway).checkout(order)
gateway.charge.assert_called_once_with(170)
```

RIESGO DETECTADO El servicio confirma el pedido antes de conocer la respuesta de la pasarela; si el pago es rechazado, el estado continúa CONFIRMED. La prueba complementaria documenta ese comportamiento actual, pero no implica que sea deseable en producción.

Mejora propuesta: separar validación y cálculo del importe de la transición irreversible de estado. Autorizar el pago primero y confirmar/persistir la orden solo cuando se haya obtenido una respuesta aprobada; si existe transacción distribuida, definir idempotencia, reintentos seguros y compensación según el contrato del proveedor.

E2–E5. Resultado específico de las nuevas pruebas

PRUEBA COMPLEMENTARIA	RESULTADO
test_vip_200_discount_170	PASSED
test_regular_200_total_200	PASSED
test_repository_250_id_2_commit	PASSED
test_gateway_rejects_vip_200_charge_170	PASSED

6. Evidencia E6 · Seguridad y autorización

La política evaluada establece esta prioridad: un pedido SHIPPED nunca puede cancelarse, incluso por administradores; para otros estados el administrador puede cancelar; el propietario no administrador puede cancelar solo antes de cumplirse 24 horas desde la creación. Se usan fechas controladas para probar exactamente el límite.

ROL / IDENTIDAD	ESTADO	EDAD	DECISIÓN
Administrador; ajeno	NEW	30 días	Permitido
Administrador; propietario	SHIPPED	0 horas	Denegado
Propietario, no admin	NEW	23 horas	Permitido
Propietario, no admin	NEW	24 horas exactas	Denegado
Propietario, no admin	NEW	25 horas	Denegado
Tercero, no admin	NEW	0 horas	Denegado

Las pruebas se encuentran en tests/test_security.py y tests/test_additional_cases.py. Una persona con rol de administrador no está exenta de la prohibición sobre envíos SHIPPED.

7. Evidencia E7 · Cobertura y calidad

Se midieron las instrucciones de src/ con pytest-cov. La cobertura lineal observada fue del 100 % (55 sentencias, ninguna omitida), por encima del umbral requerido del 80 %. El reporte navegable está en htmlcov/index.html.

E7 / resumen de cobertura — ejecución real

```
$ python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
Name                Stmts   Miss  Cover
src/application.py      8      0   100%
src/domain.py          21      0   100%
src/repository.py      18      0   100%
src/security.py         8      0   100%
TOTAL                  55      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
```

Visualización de salida real de pytest-cov; el HTML navegable se entrega en la carpeta htmlcov/.

MÓDULO	SENTENCIAS	SIN EJECUTAR	COBERTURA
src/application.py	8	0	100 %
src/domain.py	21	0	100 %
src/repository.py	18	0	100 %
src/security.py	8	0	100 %
TOTAL	55	0	100 %

La medición de 100 % no demuestra ausencia de defectos. Un riesgo de diseño todavía relevante es el pedido confirmado con pago rechazado: se documenta y prueba que ocurre, pero la cobertura de líneas por sí sola no lo corrige. Esta es la reflexión solicitada sobre una situación todavía no resuelta a nivel de consistencia de negocio.

RUFF PENDIENTE El proyecto incluye reglas Ruff y el ejecutable Windows intenta corregir problemas de estilo. En la validación de este reporte Ruff no estaba instalado, por lo que no se declara una aprobación suya. Ejecutar python -m ruff check src tests cargar_datos_demo.py y guardar la captura cuando termine sin errores.

8. Evidencia E8 · Integración continua con GitHub Actions

El proyecto ya incluye .github/workflows/ci.yml (CI Business Logic) para eventos push y pull_request. Instala Python 3.11, dependencias, ejecuta Ruff, pytest con cobertura mínima del 80 % y compileall. Esto prueba la preparación del pipeline, pero no su ejecución efectiva en una cuenta GitHub.

PASO DE LA GUÍA	EVIDENCIA REQUERIDA	ESTADO
Subir repositorio	URL pública o accesible para el docente	Pendiente
Primera ejecución	Captura verde del workflow	Pendiente
Falla controlada	Cambiar temporalmente una aserción y capturar rojo	Pendiente
Recuperación	Revertir la falla, push y capturar verde	Pendiente
Protección de rama	Configurar ruleset si se exige impedir merge	No automático

URL del repositorio: _____

Captura verde: [insertar] Captura roja: [insertar] Captura verde final: [insertar]

Explicación propuesta: el pipeline debe detectar la aserción incorrecta, marcar el trabajo como fallido e impedir dar por válida una modificación con pruebas rotas. Al restaurar la aserción y enviar otro commit, la ejecución debe volver a pasar. No debe afirmarse que GitHub bloquea el merge si no se configuró una regla de protección.

9. Evidencia E9 · Aplicación al PIA de la empresa

PENDIENTE DE DATOS DEL PIA La guía exige al menos tres reglas REALES aprobadas del proyecto integrador de tu empresa, no tres reglas del proyecto didáctico de pedidos. Como el PIA no fue adjuntado, no corresponde inventar entidades, políticas ni resultados. Se entrega una plantilla editable con nueve casos.

Empresa del PIA: _____

Entidades / Value Objects / servicios / repositorios: _____

Regla 1 (aprobada): _____

Regla 2 (aprobada): _____

Regla 3 (aprobada): _____

ID	REGLA	TIPO	ENTRADA	RESULTADO ESPERADO
PIA-01	R1	Éxito	[completar]	[completar]
PIA-02	R1	Error	[completar]	[completar]
PIA-03	R1	Límite	[completar]	[completar]
PIA-04	R2	Éxito	[completar]	[completar]
PIA-05	R2	Error	[completar]	[completar]
PIA-06	R2	Límite	[completar]	[completar]
PIA-07	R3	Éxito	[completar]	[completar]
PIA-08	R3	Error	[completar]	[completar]
PIA-09	R3	Límite	[completar]	[completar]

Agregar al menos una prueba de persistencia y una con mock si el PIA usa un servicio externo. Si no hay integración externa, justificarlo y realizar un adaptador simulado como ejercicio separado. Para E9 también debe entregarse un CSV con entradas, expectativas, resultados reales, estado y pruebas ejecutables.

Decisiones de diseño del PIA: _____

_____

9.1 Estrategia extremo a extremo (E2E) propuesta

En el proyecto base no existe API web ni interfaz, por lo que no se declara una prueba E2E ejecutada. Como escenario futuro se propone: iniciar pedido válido → verificar reglas de cliente y descuento → solicitar autorización al proveedor de pago → persistir el estado confirmado únicamente si el cargo fue aprobado → consultar el pedido.

Un segundo recorrido debe simular rechazo de pago y comprobar que no se conserva un pedido confirmado sin cobro.

10. Matriz de resultados del laboratorio (CP01–CP15)

La matriz completa con esperado, observado y estado está en MATRIZ_RESULTADOS.csv. Este resumen transcribe la última validación del proyecto didáctico. La matriz NO sustituye la matriz propia del PIA.

CASO	RESULTADO ESPERADO	RESULTADO REAL	ESTADO
CP01	CONFIRMED, cobro 120.00	CONFIRMED, cobro 120.00	APROBADO
CP02	CONFIRMED, cobro 170.00	CONFIRMED, cobro 170.00	APROBADO
CP03	CustomerHasDebt	CustomerHasDebt	APROBADO
CP04	ValueError	ValueError	APROBADO
CP05	ValueError	ValueError	APROBADO
CP06	CONFIRMED, cobro 850.00	CONFIRMED, cobro 850.00	APROBADO
CP07	CONFIRMED, cobro 0.01	CONFIRMED, cobro 0.01	APROBADO
CP08	OrderAlreadyConfirmed	OrderAlreadyConfirmed	APROBADO
CP09	CONFIRMED, cobro 255.00	CONFIRMED, cobro 255.00	APROBADO
CP10	CONFIRMED, cobro 75.00	CONFIRMED, cobro 75.00	APROBADO
CP11	CustomerHasDebt	CustomerHasDebt	APROBADO
CP12	CONFIRMED, cobro 84.99	CONFIRMED, cobro 84.99	APROBADO
CP13	RuntimeError: payment rejected	RuntimeError: payment rejected	APROBADO
CP14	Registro visible después de commit	Registro visible después de commit	APROBADO
CP15	Registro ausente después de rollback	Registro ausente después de rollback	APROBADO

CP08 confirma que no se repite una confirmación; CP13 documenta el rechazo del gateway; CP14 y CP15 verifican commit y rollback. Los 15 casos figuran como APROBADO en la matriz del proyecto.

11. Uso responsable de IA y revisión crítica

El proyecto fue elaborado con apoyo de IA y posteriormente contrastado con los requisitos del documento. Se revisó la prueba de rechazo de pago CP13: la versión recibida utilizaba un pedido regular de 50, suficiente para demostrar RuntimeError, pero no verificaba el escenario VIP de 200 con cargo de 170 expresamente solicitado en la guía. Se añadió test_gateway_rejects_vip_200_charge_170 y se comprobó, con un mock, la llamada exacta y la excepción. Este ajuste evidencia que una prueba sugerida o generada con IA debe revisarse frente a la regla aprobada y sus límites, no aceptarse solo porque pasa.

12. Reflexión final (150–200 palabras)

El defecto de negocio más costoso que podría llegar a producción sería confirmar un pedido cuando el pago ha sido rechazado. Esta inconsistencia permite que el sistema trate una venta como completada sin haber recibido dinero, lo que podría provocar entregas no cobradas, conflictos con clientes y errores contables. En el proyecto analizado, el servicio confirma el pedido antes de llamar a la pasarela; por eso, aunque el mock devuelve una respuesta negativa y se lanza una excepción, el estado ya quedó marcado como CONFIRMED. La prueba que ayuda a detectar el riesgo es la de pago rechazado para un cliente VIP con total de 200: comprueba que se intenta cobrar 170, que ocurre el rechazo y que el pedido permanece confirmado. Para prevenir realmente el defecto, propondría cambiar el flujo de negocio de modo que la confirmación definitiva se produzca únicamente después de una autorización exitosa y que la persistencia se haga dentro de una transacción controlada. Además, mantendría pruebas de regresión para rechazo, reintentos y confirmación duplicada, junto con el pipeline de integración continua.

Extensión: 176 palabras. Texto propuesto para revisar y personalizar.

13. Lista de comprobación antes de entregar

ENTREGABLE	VERIFICACIÓN / ACCIÓN
Código src/ y tests/	Incluido, con 37 pruebas aprobadas
Matriz de resultados	MATRIZ_RESULTADOS.csv incluido (15 casos del laboratorio)
Cobertura HTML	htmlcov/index.html generado; 100 % de src/
RED y GREEN TDD	Registros de ejecuciones reales disponibles
Mock / commit / rollback	Pruebas incluidas y registradas
README y ejecutables	Incluidos para instalación y ejecución
Ruff	Ejecutar y capturar resultado en la PC
CI GitHub verde/rojo/verde	Subir repositorio y generar capturas reales
E9: PIA de empresa	Completar 3 reglas, 9 casos, código y cobertura
Datos privados / .venv	No incluir tokens, claves, venv ni credenciales

Referencias del trabajo

Guía del curso: laboratorio_guiado_unidad_5_cln_corregido.docx, secciones 1–14 (documento proporcionado para la actividad).